juliacon

Enhancing Deterministic Voice Control with Safe LLM Interaction

Samuel Omlin[1]

[1]Swiss National Supercomputing Centre (CSCS), ETH Zurich, Lugano, Switzerland

ABSTRACT

Small desktop tasks benefit from speech and large-language-model assistance, but open-ended agents relinquish action control while text chat keeps interaction cumbersome. This paper presents an approach that extends deterministic voice control with dual speech recognition, text-to-speech, and clipboard- or selection-aware large-language-model interaction while keeping system actions under deterministic command control. The implementation couples predefined commands with seven composable primitives and application-specific command dictionaries, allowing concise definitions that range from spoken follow-up chat to Jupyter-specific assistance. The resulting interaction enables seamless and safe integration of large-language-model intelligence into everyday PC workflows, more efficient application-specific prompting, and almost human-like follow-up through spoken responses, thereby helping bridge a gap in human-AI interaction.

Keywords

deterministic voice control, large language models, speech interfaces, programmable voice assistants, human-AI interaction, desktop automation

1. Introduction

Seamless and safe integration of LLM intelligence into small, everyday tasks on a PC remains insufficiently served. One adjacent class is voice assistants whose functionality is exposed through predefined intents, sample utterances, and skill interfaces rather than programmable application-specific interaction tailored to a user's active workflow [11]. Another adjacent class is AI agents and other LLM-based systems that act through tool use. Recent published work on autonomous agent safety emphasizes that such agents operate over personal information and device settings, can cause harmful side effects, and require explicit safety mechanisms, robust safeguards, and user trust [2, 1]. In that design space, per-action approval quickly becomes cumbersome, and the operational temptation is to slide toward unsafe yolo-style autonomy instead of keeping actions under deterministic control. A third adjacent class is plain chat-based interaction. Multitasking dialogue research treats switching between conversation and real-time tasks as a core difficulty, and chat-based conversational agents require explicit mechanisms for concurrent conversation tracking and memory even to manage context across interactions [10, 3]. The niche addressed here is therefore not a closed assistant capability catalog, not unrestricted tool autonomy, and not stand-alone chat, but deterministic voice control extended with LLM interaction for small desktop workflows, together with application-specific commands that make that interaction more efficient while keeping system actions explicit and safe.

This paper presents an approach for 1) integrating LLM interaction into a deterministic voice-control system so that LLM intelligence is available for small everyday tasks while system actions remain under safe deterministic command control; and 2) application-specific LLM interaction that is safe and programmable. The approach was successfully implemented in JustSayIt.jl.

2. Approach

The approach extends the deterministic foundation described in the 2024 JustSayIt paper [6] and JuliaCon 2023 talk [5] rather than recapping it. Command recognition continues to use a constrained low-latency path, while free-speech interaction is handled by a separate higher-capacity path. In the current implementation, command recognition uses Vosk [7] and free-speech recognition uses faster-whisper [9].

Three additions bring LLM interaction into this deterministic setting. First, spoken LLM replies are supported through integrated text-to-speech, enabling follow-up exchanges that feel conversational without relinquishing control of system actions. Second, selected text and clipboard content are available as immediate context sources, so short desktop tasks can be driven from what the user is already editing or reading. Third, predefined LLM, speech, text-access, and speech-output commands are complemented by a small base-language-like primitive layer, plus lower-level LLM access for programmable voice workflows.

3. Implementation and Results

The approach is implemented in JustSayIt.jl[1]. LLM execution combines Ollama [4] with PromptingTools.jl [8]; the dual STT path uses Vosk [7] for constrained commands and faster-whisper [9] for free speech. This yields deterministic control over actions while allowing free-form prompting, follow-up interaction, and context-aware responses that can be typed back into the active application or spoken aloud.

Table 1 lists the seven root-level primitives that support direct composition. Code 1 shows the minimal chatbot case: a spoken follow-up loop is defined entirely by composing recognition, LLM follow-up, and speech output. Code 2 shows the complementary route for focused workflows. Instead of defining new low-level functions, an application-specific command dictionary reuses predefined LLM commands to turn spoken instructions into Jupyter magics or to explain selected notebook code with clipboard or selection context.

[1]Release target for the presented contribution: v0.4.0, `https://github.com/omlins/JustSayIt.jl/releases/tag/v0.4.0`.

Table 1. : Composable primitives for LLM interaction, voice input, text capture, and typed or spoken output.

`listen`	Convert speech to text
`take`	Get clipboard content
`grab`	Get selected text
`ask`	Query LLM
`ask!`	Follow up on previous LLM query
`type`	Type text at cursor
`say`	Convert text to speech

Code 1: Minimal voice chatbot defined by direct composition of speech input, LLM follow-up, and spoken output.

```
llm_commands = Dict("chatbot" => () ->  while true
                        listen() |> ask! |> say
                    end)
```

Code 2: Application-specific Jupyter commands built from predefined LLM commands. In this example, `LLM.type_answer` answers an uttered instruction directly, whereas `LLM.type_text_answer_to` answers with respect to selected text or clipboard content; the first command types an IPython magic for a spoken task and the second types a markdown explanation of the selected cell.

```
jupyter_llm_commands = Dict(
    "generate magic" => () -> LLM.type_answer(instruction_prefix
     ="Generate the appropriate IPython magic command for the
     following task using proper syntax (%line magic or %%cell
     magic) with necessary options:"),
    "explain cell" => () -> LLM.type_text_answer_to("Explain
     what the selected
    cell code does in simple terms. Include key functions,
     algorithms, and data
    transformations. Format as markdown with clear section
     headings."),
)
```

4. Conclusions

This approach enables seamless and safe integration of LLM intelligence into small, everyday PC tasks while preserving deterministic control over system actions. Application-specific command dictionaries make interaction more efficient for focused workflows, and TTS-backed follow-up makes the exchange almost human-like. Together, these mechanisms bridge a practical gap in human-AI interaction between deterministic desktop control and flexible language-model assistance.

5. References

[1] Jason Jabbour and Vijay Janapa Reddi. Generative AI agents in autonomous machines: A safety perspective. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–13, 2025. doi:10.1145/3676536.3698390.

[2] Juyong Lee, Dongyoon Hahm, June Suk Choi, W. Bradley Knox, and Kimin Lee. Mobilesafetybench: Evaluating safety of autonomous agents in mobile device control. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(44):37565–37573, 2026. doi:10.1609/aaai.v40i44.41090.

[3] Victor R. Martinez and James Kennedy. A multiparty chat-based dialogue system with concurrent conversation tracking and memory. In *Proceedings of the 2nd Conference on Conversational User Interfaces*, pages 1–9. ACM, 2020. doi:10.1145/3405755.3406121.

[4] Ollama. Ollama. Project website, 2026. `https://ollama.com/`.

[5] Samuel Omlin. Quick assembly of personalized voice assistants with justsayit. JuliaCon 2023 talk abstract and video, 2023. `https://pretalx.com/juliacon2023/talk/review/9MJFPDJV9DR7ANUXPSP9ZWJRFWSE83EY`.

[6] Samuel Omlin. Justsayit.jl: A fresh approach to open source voice assistant development. *The Proceedings of the JuliaCon Conferences*, 6(66):121, 2024. doi:10.21105/jcon.00121.

[7] Nickolay V. Shmyrev and contributors. Vosk speech recognition toolkit. Project website, 2020. `https://alphacephei.com/vosk/`.

[8] svilupp. Promptingtools.jl. Project documentation and README, 2025. `https://github.com/svilupp/PromptingTools.jl`.

[9] SYSTRAN. faster-whisper. Project documentation and README, 2025. `https://github.com/SYSTRAN/faster-whisper`.

[10] Fan Yang, Peter A. Heeman, and Andrew Kun. Switching to real-time tasks in multi-tasking dialogue. In *Proceedings of the 22nd International Conference on Computational Linguistics - COLING '08*, volume 1, pages 1025–1032. Association for Computational Linguistics, 2008. doi:10.3115/1599081.1599210.

[11] Nan Zhang, Xianghang Mi, Xuan Feng, XiaoFeng Wang, Yuan Tian, Feng Qian, et al. Dangerous skills: Understanding and mitigating security risks of voice-controlled third-party functions on virtual personal assistant systems. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1381–1396, 2019. doi:10.1109/SP.2019.00016.